# Latent Regimes in Returns and Volatility: Identification and Inference

# Why this Matters

- **Structural Stability**
  - Parameters vary through time (e.g., volatility, beta, correlation)
  - **Single “average” model can be misleading**

- **Asymmetry**
  - Downturn markets  mostly behave differently than Up markets
    - Asymmetric beta
    - Correlations rise in downturns
    - Tail risk is one sided – downside has fatter tails

- **Risk Concentration During Transitions**
  - Losses potentially increase during system *moves* from one regime to another, not while it sits in a stable state

# Where this Matters

- Volatility forecasting
- Tactical asset asset allocation
- Stress period identification
- Scenario generation
- Other..

# The Data: Obtain, Resample and Build Features

```python
data = pd.read_excel('../ParkingLot_Section3/SomeFile.xlsx')
data['Date'] = pd.to_datetime(data['Date'])
data = data.set_index('Date').rename(columns={'USLC': 'Returns'}).dropna(subset=['Returns'])

r_daily       = data['Returns'].astype(float).dropna()
weekly_simple = (1.0 + r_daily).resample('W-FRI').prod() - 1.0
weekly_log    = np.log1p(weekly_simple).dropna()

w = pd.DataFrame(index=weekly_log.index)
w['r1w']      = weekly_log                        # 1-week log return
w['r13w']     = w['r1w'].rolling(13).sum()        # ~quarterly
w['r26w']     = w['r1w'].rolling(26).sum()        # ~semi-annual
w['r52w']     = w['r1w'].rolling(52).sum()        # ~annual
w['mom4w']    = w['r1w'].rolling(4).sum()         # 4-week momentum

price_idx     = (1.0 + weekly_simple).cumprod()
w['drawdown'] = price_idx / price_idx.cummax() - 1.0  # drawdown from peak
w['price']    = np.exp(w['r1w'].cumsum())              # cumulative log-price index

w['trend52'] = w['r1w'].rolling(52).sum()
w = w.dropna().copy()

print(f'{len(w)} weekly observations  ({w.index[0].date()} → {w.index[-1].date()})')
print()
print(w[['r1w', 'r13w', 'r52w', 'drawdown', 'trend52']].describe().round(4))
```

trend52 - is a config-agnostic anchor that ensures "Bull" means the same thing across all four feature configurations — making the cross-config comparisons meaningful

# Build Model – Feature Set Configuration as Controll Experiment Harness

**Enable** to execute **same regime-switching pipeline** multiple times with **different subset of features**

```
CONFIGS = {
    'Directional\n(r1w, r13w, r52w)':   ['r1w', 'r13w', 'r52w'],
    'Drawdown\n(r1w, r13w, drawdown)':  ['r1w', 'r13w', 'drawdown'],
    'Momentum\n(r1w, mom4w, r26w)':     ['r1w', 'mom4w', 'r26w'],
    'Single feature\n(r52w only)':      ['r52w'],
}
```

- **Directional :** Market going up ↑ or down ↓, consistently over the short, medium, and long term
- **Drawdown :** What's the trend, and how far below the last peak
- **Momentum :** What's the trend, and is it carrying through - move have follow-through or is it stalling/reversing
- **Single Feature :** A simple long-term yardstick

# Build Model – Iterate Through

```python
# Standardise selected features from CONFIGS.items()
X = StandardScaler().fit_transform(w[feature_cols].values)

# Covariance_type='full' - allow for correlations between features
model = hmm.GaussianHMM(
    n_components=n_states,
    covariance_type='full',
    n_iter=500,
    random_state=random_state
)
model.fit(X)

# Posterior regime probabilities per time step
post = model.predict_proba(X)  # shape: (T, n_states)

# --- UNIVERSAL labeling anchor (bear = negative trend) ---
anchor = 'trend52'
if anchor not in w.columns:
    raise KeyError(
        "Missing universal anchor 'trend52'. Create it before fitting:\n"
        "w['trend52'] = w['r1w'].rolling(52).sum(); w = w.dropna().copy()"
    )

# Posterior-weighted mean anchor per state (more robust than argmax grouping)
anchor_vals = w[anchor].values
state_means = {s: np.average(anchor_vals, weights=post[:, s]) for s in range(n_states)}
means = pd.Series(state_means).sort_values()

bear_s, neutral_s, bull_s = map(int, means.index.tolist())

# Output probabilities mapped to labelled states
out = pd.DataFrame(index=w.index)
out['p_bull']    = post[:, bull_s]
out['p_neutral'] = post[:, neutral_s]
out['p_bear']    = post[:, bear_s]
out['regime']    = (
    out[['p_bull', 'p_neutral', 'p_bear']]
    .idxmax(axis=1)
    .str.replace('p_', '', regex=False)
    .str.capitalize()
)
```

- Standardize all the columns submitted
- Create Hidden Markov Model – x3 states
- For each regime, allows the features to be correlated
- Random_state reproducible

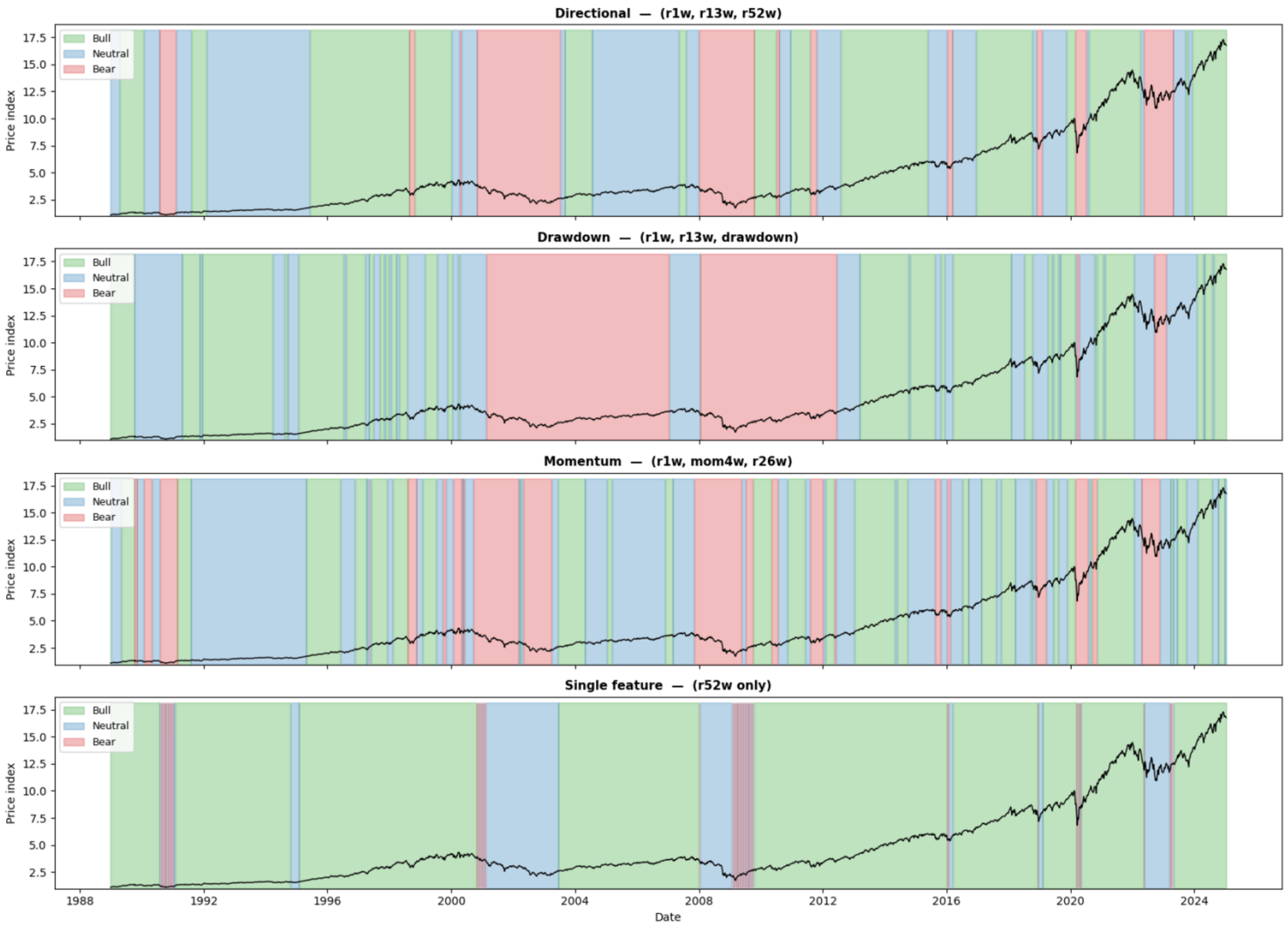
- trend52 is 52-week rolling sum of 1-week log returns
- HMM only finds hidden states statistically
- Does not know which one should be called “bear”
- **Common reference for deciding which hidden state (bear, bull etc.)**

# Build Model – Iterate Through Model Configurations (output)

| | Directional\n(r1w, r13w, r52w) | | | | Drawdown\n(r1w, r13w, drawdown) | | | |
|---|---|---|---|---|---|---|---|---|
| | p_bull | p_neutral | p_bear | regime | p_bull | p_neutral | p_bear | regime |
| Date | | | | | | | | |
| 1988-12-30 | 1.639669e-86 | 1.000000e+00 | 0.000000e+00 | Neutral | 1.000000 | 0.000000e+00 | 0.000000 | Bull |
| 1989-01-06 | 1.972002e-04 | 9.997917e-01 | 1.111889e-05 | Neutral | 0.999895 | 7.103214e-45 | 0.000105 | Bull |
| 1989-01-13 | 1.662027e-04 | 9.998284e-01 | 5.427922e-06 | Neutral | 0.999956 | 8.775666e-14 | 0.000044 | Bull |
| 1989-01-20 | 4.991509e-04 | 9.994870e-01 | 1.385566e-05 | Neutral | 0.999852 | 6.229467e-14 | 0.000148 | Bull |
| 1989-01-27 | 3.997627e-04 | 9.995915e-01 | 8.769693e-06 | Neutral | 0.999943 | 3.740732e-14 | 0.000057 | Bull |
| ... | ... | ... | ... | ... | ... | ... | ... | ... |
| 2024-12-13 | 9.999997e-01 | 1.220410e-07 | 1.280797e-07 | Bull | 0.999942 | 1.371359e-13 | 0.000058 | Bull |
| 2024-12-20 | 9.999991e-01 | 5.924097e-07 | 2.959923e-07 | Bull | 0.999861 | 1.296699e-13 | 0.000139 | Bull |
| 2024-12-27 | 9.999984e-01 | 1.457200e-06 | 1.883052e-07 | Bull | 0.999878 | 6.104680e-13 | 0.000122 | Bull |
| 2025-01-03 | 9.999991e-01 | 4.636571e-07 | 4.217531e-07 | Bull | 0.999636 | 1.695906e-11 | 0.000364 | Bull |
| 2025-01-10 | 9.998380e-01 | 1.137765e-04 | 4.821851e-05 | Bull | 0.995214 | 1.975348e-08 | 0.004786 | Bull |

1881 rows × 16 columns

# Regime Shading by Feature Configuration




## Directional (r1w, r13w, r52w)

- Sustained **positive drift** across all horizons
- Tends to be **moderately stable**: not as smooth as r52w-only

## Drawdown (r1w, r13w, drawdown)

- Is the market is **near its recent peak** or still **well below it**
- **Is the market still recovering from a past sell-off**

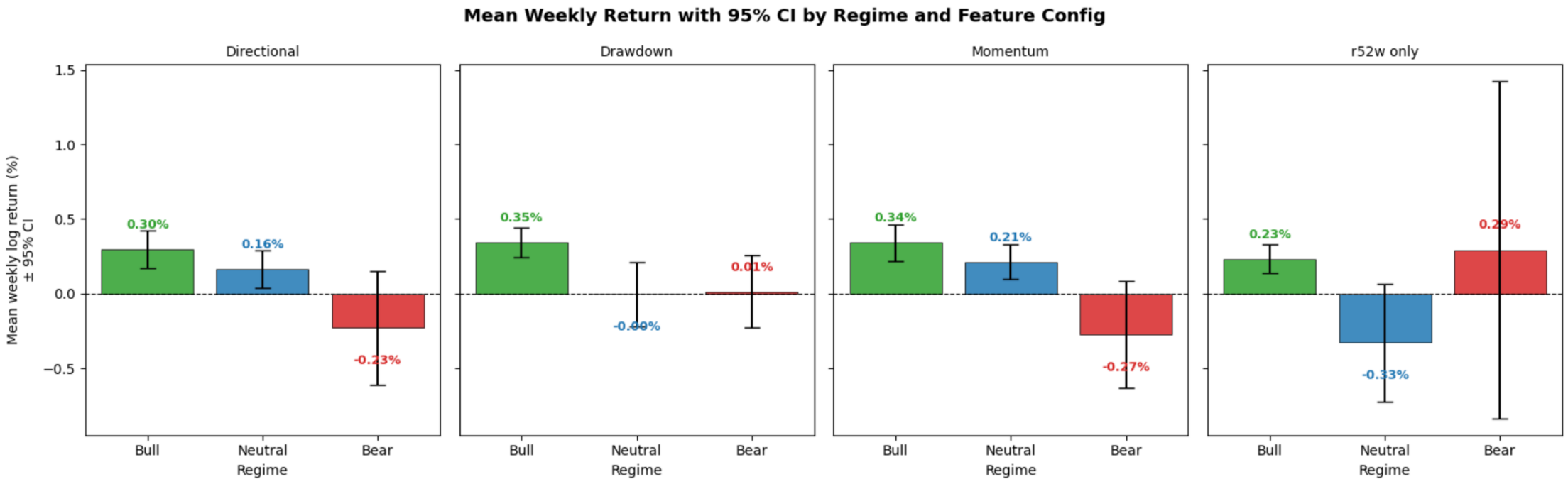
## Momentum (r1w, mom4w, r26w)

- Momentum changes quickly
- Model pick up noise rather than long structure

## Single feature (r52w only)

- Long Bull Market - baseline

# Regime Analysis Quality Check



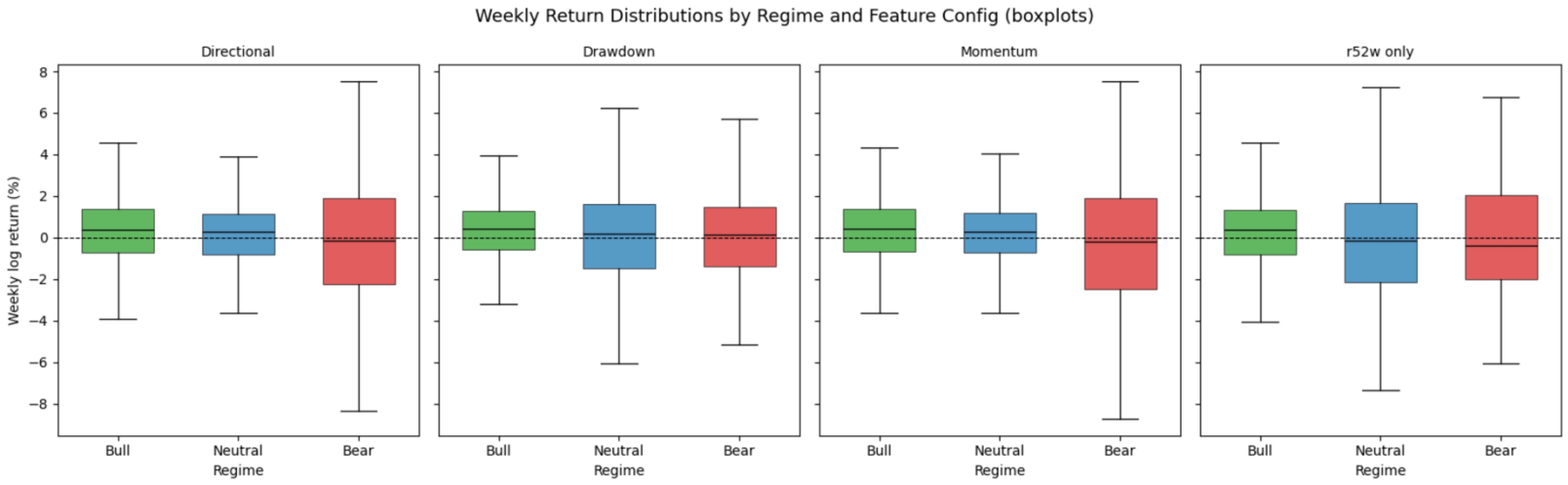

- **Check: Does** HMM regimes produce **meaningfully different** **mean returns**

  **Directional**

  - Bull: about **+0.30%**
  - Neutral: about **+0.16%**
  - Bear: about **−0.23%**

- **Directional** and **Momentum** Config look the most credible

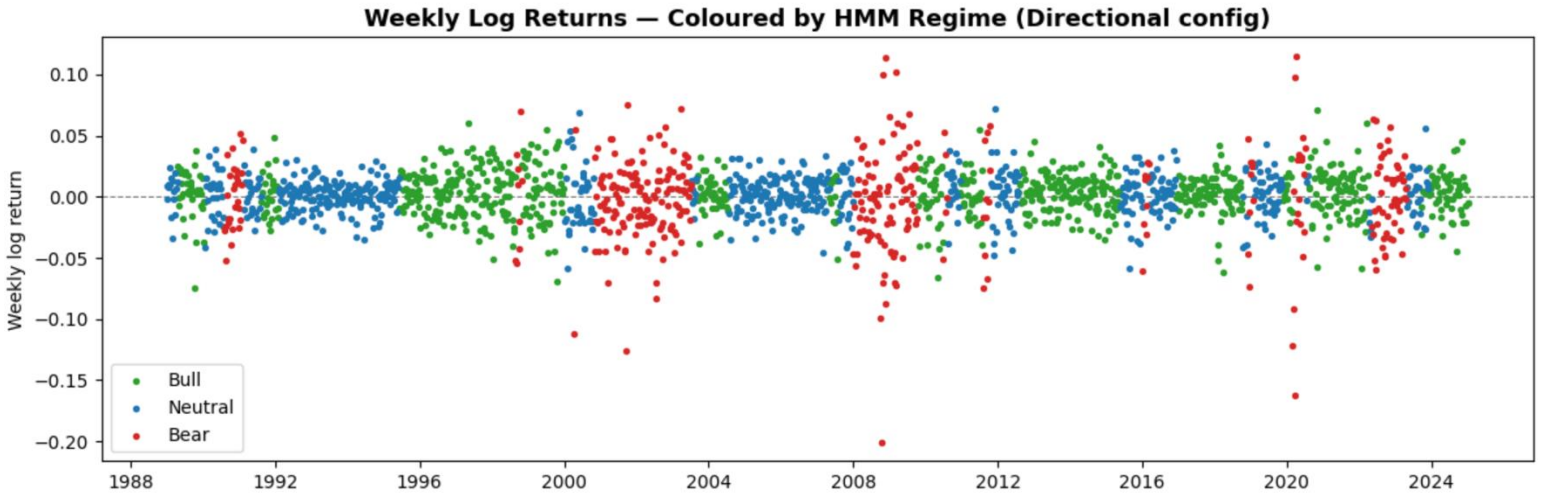
# Regime Analysis Quality Check

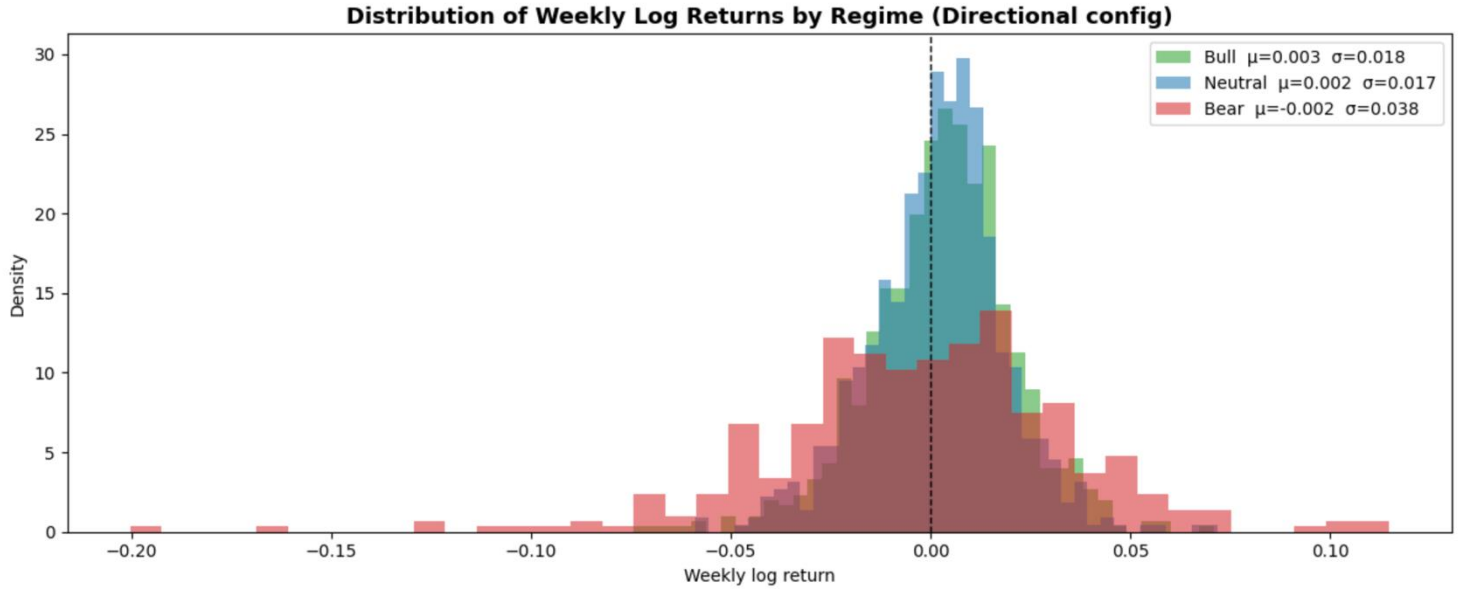



**Full distribution shape within each regime**

- Bear state is not just “slightly negative on average” — it is a **higher-risk, higher-volatility regime with larger downside realizations**

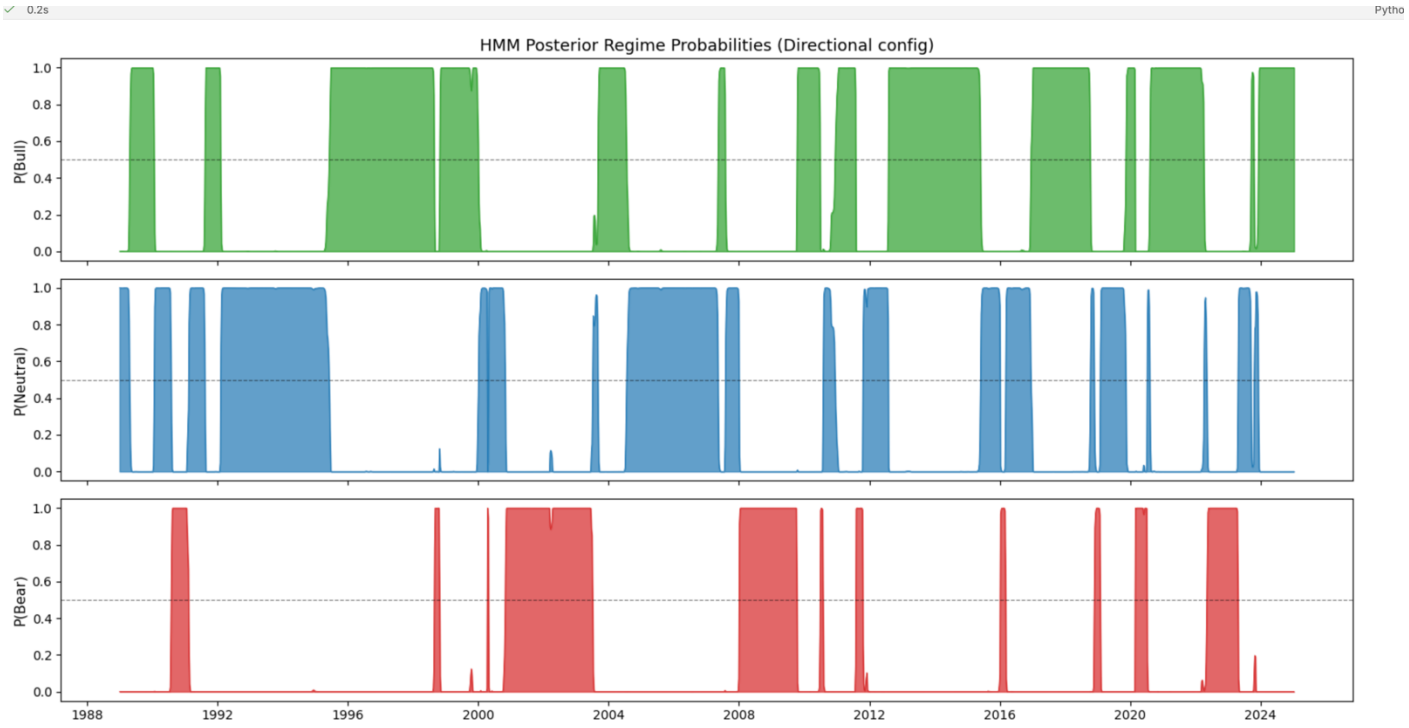
# Weekly Log Returns – by Regime (Directional Config)



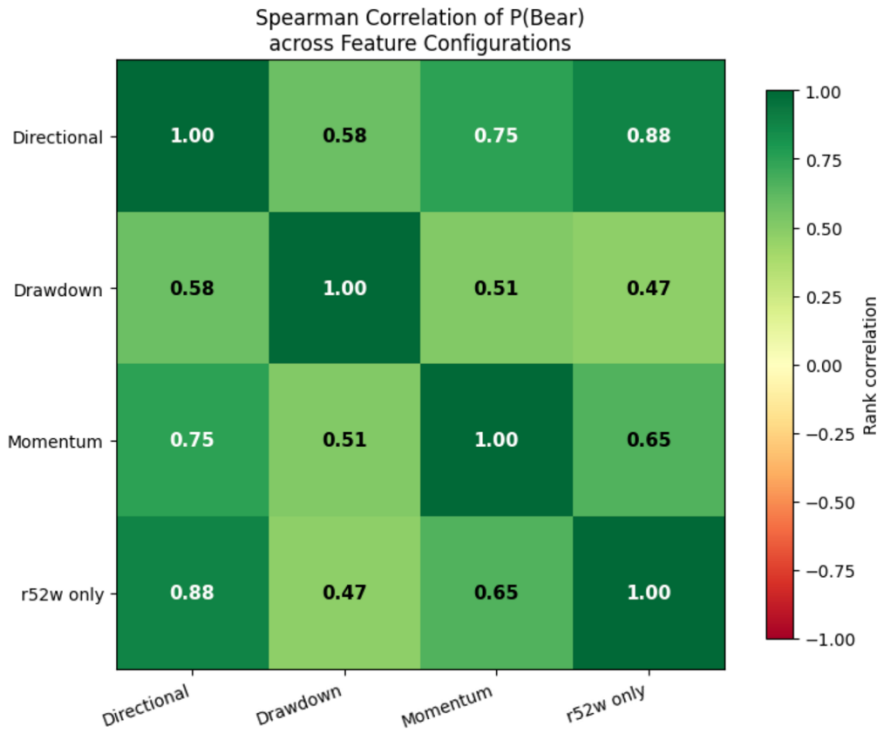

- Market returns generated by different underlying states with different behaviour
- Regimes also differ in their **return distributions**
- Bull is slightly more positive & centered round zero
- Neutral centred near zero
- Bear has a wider spread and more extreme downside outcomes – stress regime
- **Regimes correspond to different conditional distributions of returns and the whole sample can not be treated as homogeneous!**

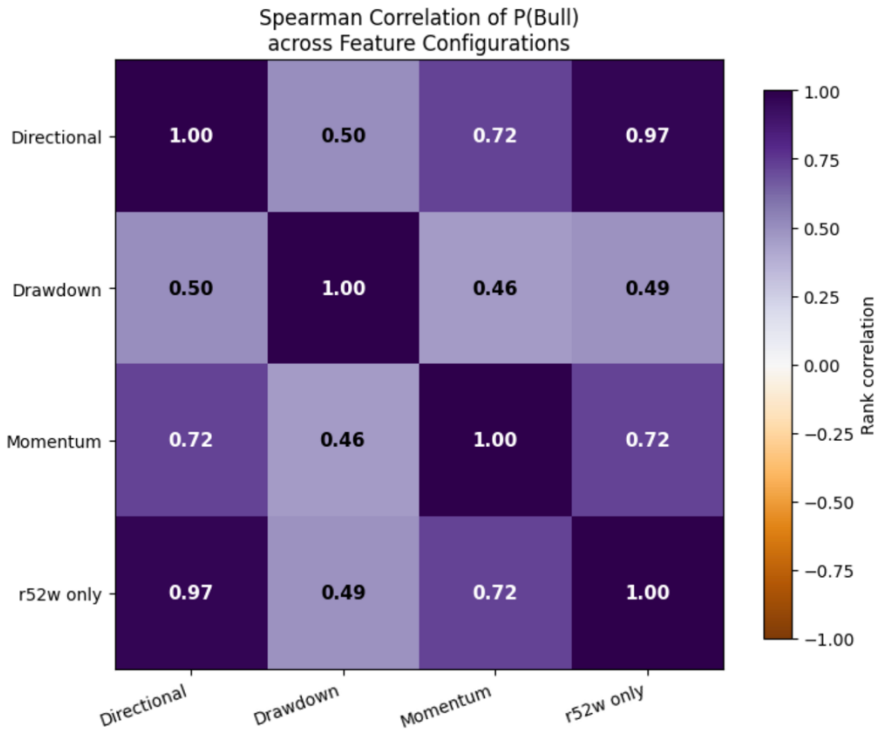
# Posterior Regime Probabilities (Directional Config)




- Posteriors are often **near 0 or near 1**, means the **HMM** is frequently making **confident state assignments** rather than constantly mixing states
- **Persistent blocks** rather than pure one-point noise, so the model is finding **state duration**, not just re-labelling each week independently
- Probabilities appear to **take turns dominating**, this seems sensible 3-state model

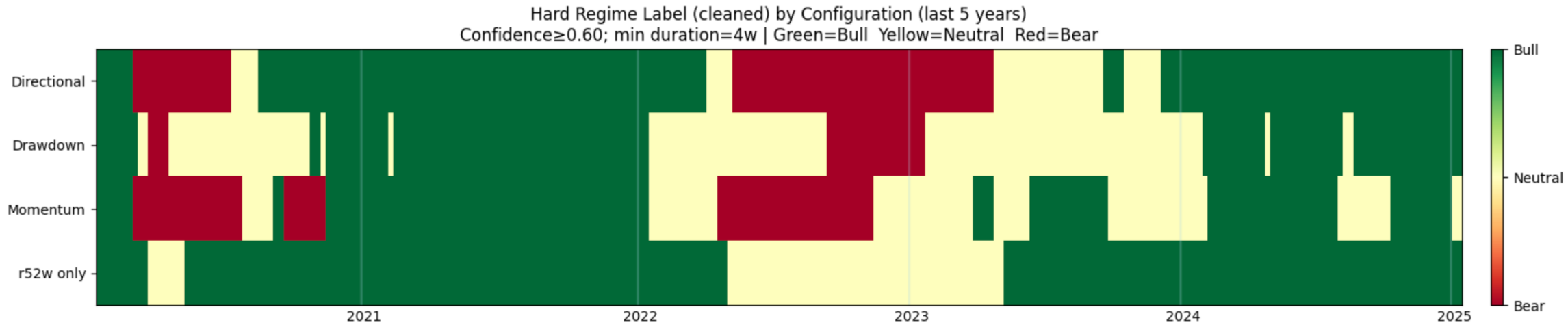
# Robustness of Regime Probabilities Across Feature Sets



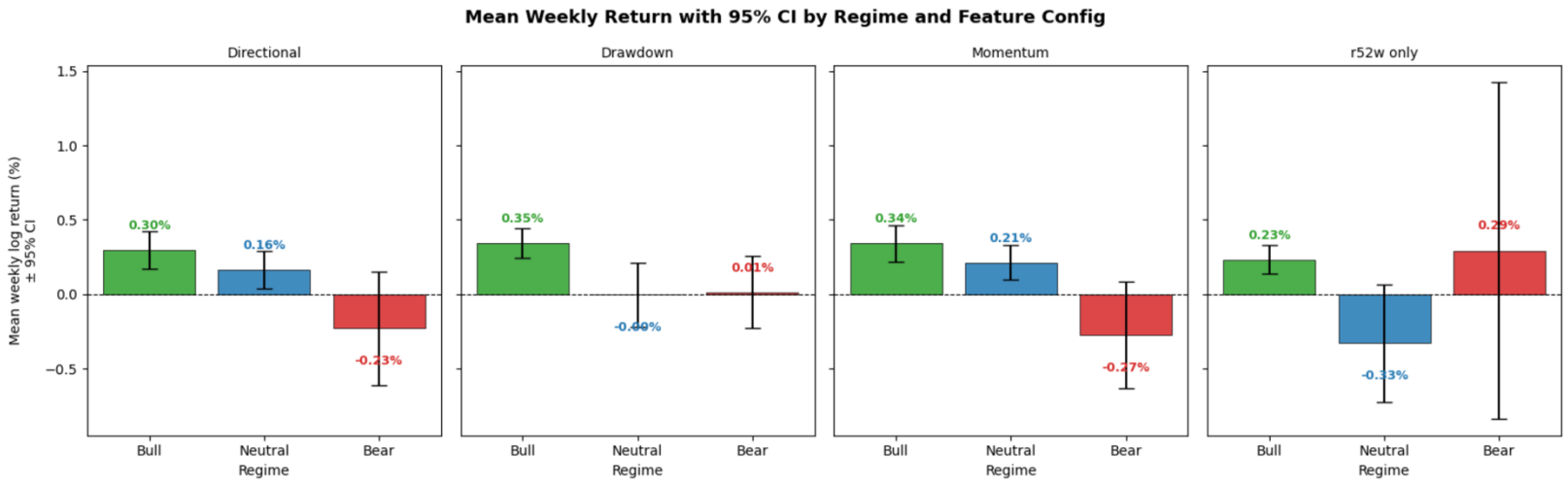

- **Directional, Momentum, and r52w-only** - tightly aligned
- **Drawdown & Bear** - move in same direction
- **Drawdown** - better aligned with Bear than Bull market

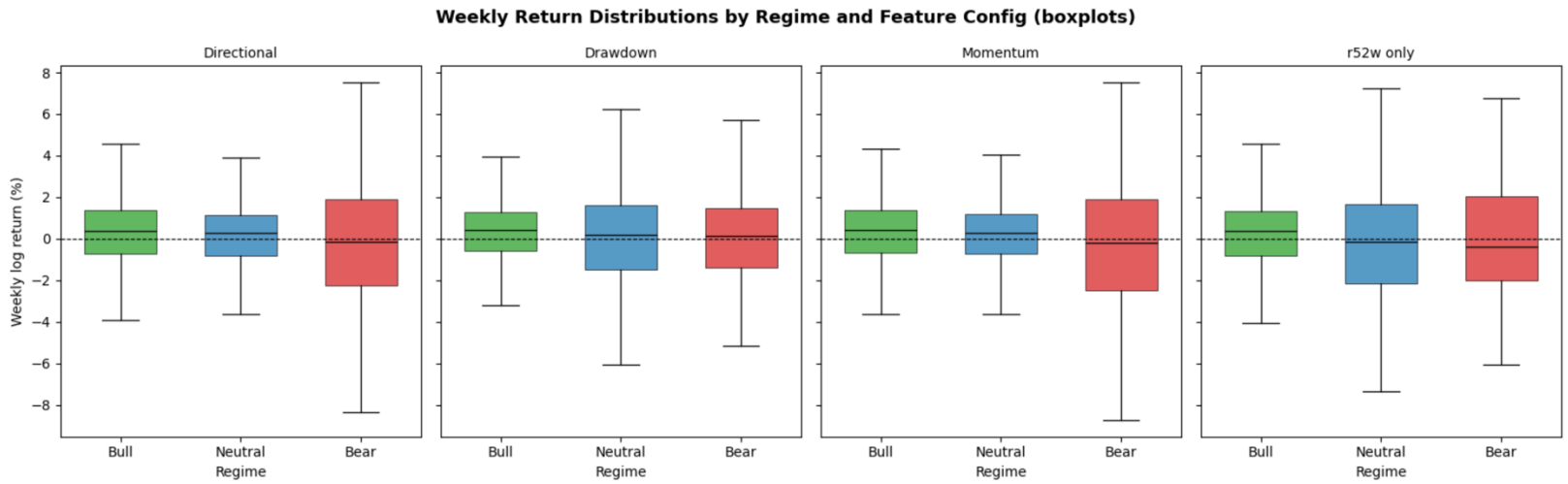
# Regime Agreement Heatmap (Last 5 Years)




- **Each row** = one feature configuration (Directional, Drawdown, Momentum, r52w-only)
  - **Sharp Bear/Neutral** episode around 2020
  - **Strong Bull** regime through much of 2021
  - **Broader Bear/Neutral** phase through ~2022–early 2023
  - **Directional and r52w-only are smoother (long-cycle trend**)

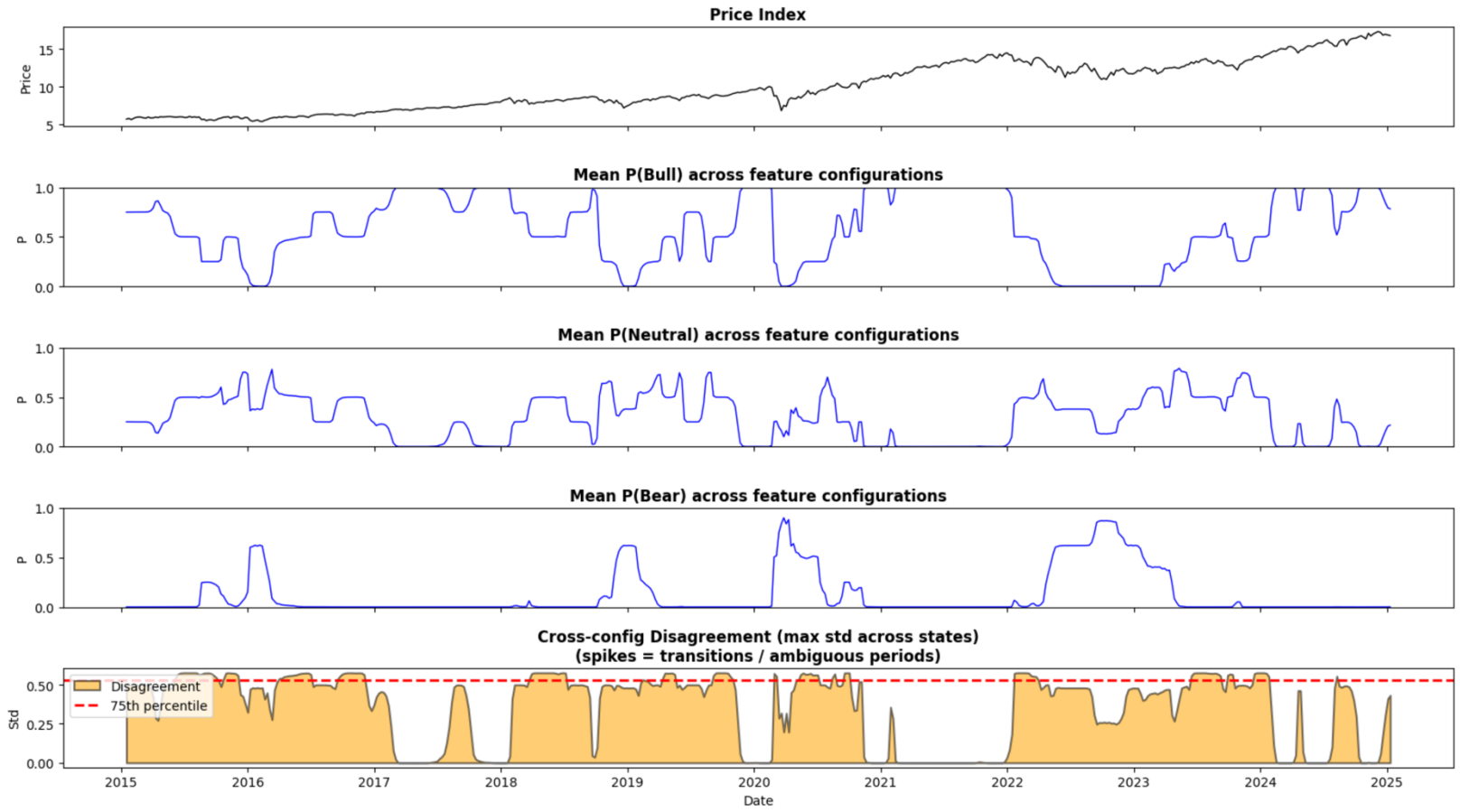
# Regime Return Profiles Across HMM Feature Configurations




- Across **most feature configurations**, **Bull regimes have the strongest average weekly returns** and **Bear regimes the weakest**, but the separation is **not equally clean** in every configuration

# Cross-Config Disagreement Score - Confidence Overlay




**y-axis : Cross-model Dispersion**

- **std** of Regime Probabilities

- **Disagreement = cross-configuration dispersion in regime probabilities**
- **Example interpretation**
  - In **2019**, the **regime signal becomes more fragile**: **Bear pressure increases**, **Bull weakens**, and model disagreement; ambiguous transition

# The Gaussian HMM: Model Structure - Summary

- **Latent regimes** an unobserved state (**Bull/Neutral/Bear**) drives the observed return behaviour each week

- **Markov switching** states evolve via a fixed **transition matrix** (probability of moving from one regime to another)
- **Gaussian emissions-** within each state, returns/features are modelled as **multivariate Gaussian** with its own **mean vector** and **full covariance**

- **State naming** label states by **average 52-week log return** (trend52): lowest = **Bear**, highest = **Bull**.

- **Fitting (EM)** iterates between
  - Compute how likely each hidden state was at each time step
  - Recompute each regime's mean/covariance and switching probabilities

- **Output: posterior probabilities** $P(S_t \mid \chi_{1:T})$ a *soft* regime signal plus hard label via argmax
  - ***S*** **-** latent regime state at time t
  - ***T*** **-** use all observed data

# x5 Week Forward Probability of Being in Each Regime (All Configurations)

**Propagate forward from the last posterior**

- Each feature configuration fits its own HMM, therefore has its own **transition matrix A** (the Markov switching dynamics).
- At the latest time ***T***, the model provides a **posterior state vector:**
  - $\boldsymbol{\pi_T = [P(S_T = Bear),\ P(S_T = Neutral),\ P(S_T = Bull)]}$

- Forecast ***h*** weeks ahead – propagate the distribution through the chain:

  - $\boldsymbol{\pi_{T+h}} = \boldsymbol{\pi_T}\mathrm{A}^{\mathrm{h}}$ where $\mathrm{A}_{\mathrm{ij}}$ is the probability of moving from state *i* to state *j*

  - $\boldsymbol{\pi_{T+h}}$: Forecast regime probability ***h*** weeks ahead with NO NEW DATA

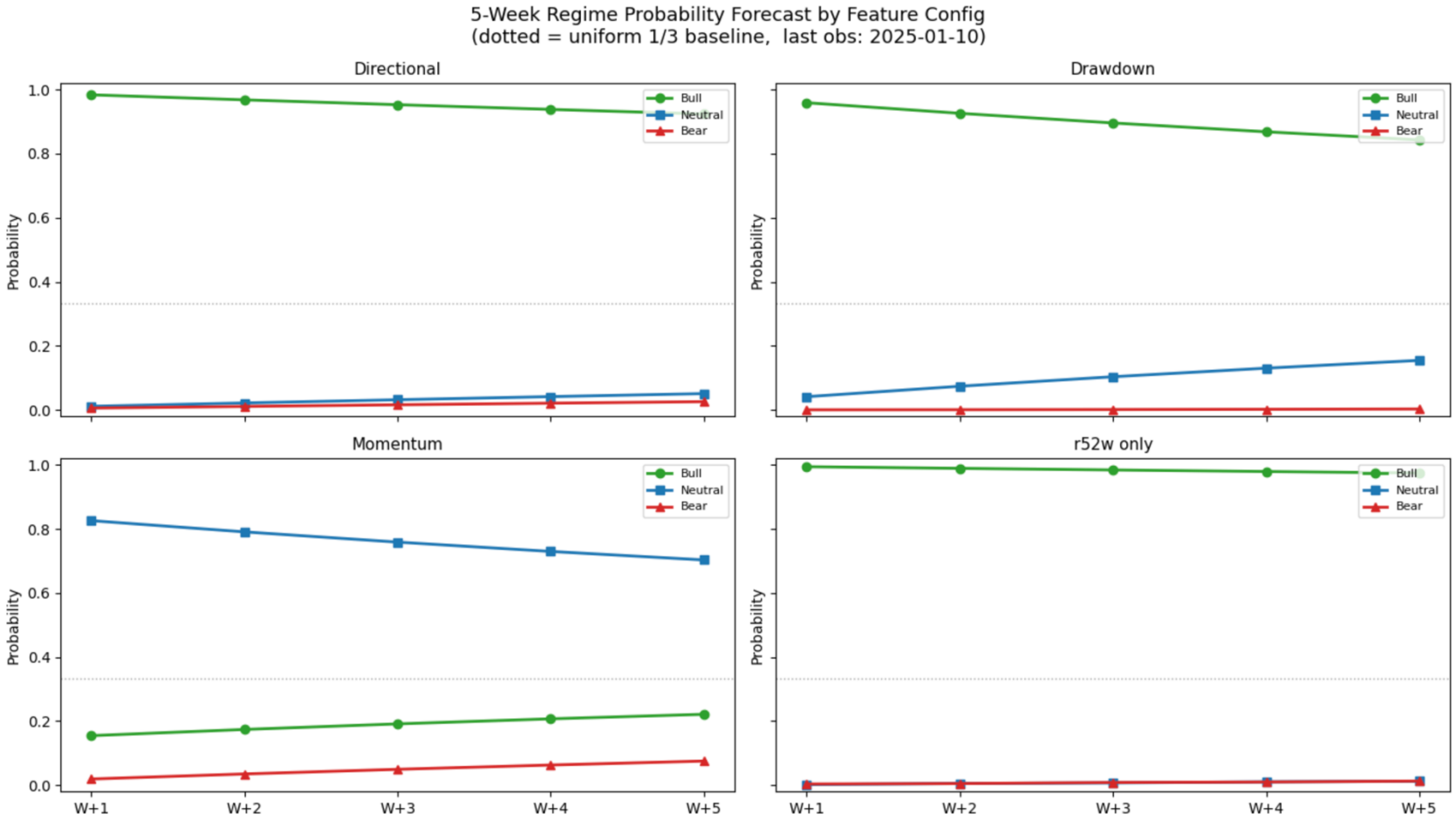
# Forecast 5 Periods Forward – Building Model

```python
forecasts = {}
weeks = np.arange(1, FORECAST_WEEKS + 1)

for name, res in results.items():
    mdl = models[name]
    A   = mdl.transmat_
    cols = CONFIGS[name]

    # --- consistent state labeling: universal trend anchor + posterior-weighted means ---
    # anchor = 'trend52'
    X = StandardScaler().fit_transform(w[cols].values)
    post = mdl.predict_proba(X)

    anchor_vals = w[anchor].values
    state_means = {s: np.average(anchor_vals, weights=post[:, s]) for s in range(3)}
    means = pd.Series(state_means).sort_values()
    bear_i, neutral_i, bull_i = map(int, means.index.tolist())

    # Last posterior from results (semantic order)
    p0 = np.array([res['p_bear'].iloc[-1], res['p_neutral'].iloc[-1], res['p_bull'].iloc[-1]])

    # Map to internal order
    p0_internal = np.zeros(3)
    p0_internal[bear_i]    = p0[0]
    p0_internal[neutral_i] = p0[1]
    p0_internal[bull_i]    = p0[2]

    # h-step propagation
    fp_list = []
    p = p0_internal.copy()
    for _ in range(FORECAST_WEEKS):
        p = p @ A
        fp_list.append(p.copy())
    fp = np.vstack(fp_list)

    # Map back to semantic order for reporting
    forecasts[name] = {
        'p_bull':    fp[:, bull_i],
        'p_neutral': fp[:, neutral_i],
        'p_bear':    fp[:, bear_i],
    }
```

***p = p @ A***

- *p – vector of prob*
- *A – Transition Matrix*
- *@ - matrix-multiply*

# 5 Week Regime Probability Forecast by Feature Configuration




**5-week regime STATE forecast (Not Market): Transition driven projection, no new information, basis = Current Posterior State**

- **Directional** and **r52w-only** indicate a highly persistent **Bull** regime, downdrift reflects mean reversion
- **Drawdown** remains **Bull** but **drifts toward Neutral**
- **Momentum is Neutral-heavy,** current regime - most likely neutral, Neutral decays , potential migration to Bull
- Etc.

# Cross-Check: Markov Switching Regression

## Markov Switching Regression (MSR)

- A univariate regime model in which returns switch between latent states, each with its own **mean** and **variance**, with switching governed by a Markov process

## Motivation

- **Independent validation model** on the same return series
- Use only **weekly log returns**, unlike the multi-feature HMM
- Estimated by **maximum likelihood via the Hamilton filter (**infer an unobserved regime or state from observed data in **MSR)**
- Detects **statistical breaks in the return process itself**
- **HMM captures directional geometry across multiple return horizons**
- **MSR captures regime changes directly in the return-generating process**

## Key Diagnostic

- **Disagreement score =** $$D_t = \frac{1}{3} \sum_{s \in \{bull, neutral, bear\}} \left| p_{s,t}^{HMM} - p_{s,t}^{MSR} \right|$$

## Objective

- Compare the full probability vectors from HMM and Markov Switching Regression

## Fit the Model

```python
# ── Fit Markov Regression ──────────────────────────────────────────────
markov_series = w['r1w'].copy()
markov_series.index = pd.DatetimeIndex(markov_series.index, freq='W-FRI')

ms_model  = MarkovRegression(markov_series, k_regimes=3, trend='c', switching_variance=True)
ms_result = ms_model.fit(search_reps=10, search_iter=10, disp=False)

print(ms_result.summary())
```

**log_returns**

- the dependent variable being modeled, here weekly returns

**k_regimes=3**

- estimate 3 latent regimes like bear / neutral / bull

**trend='c'**

- includes a constant term in each regime facilitates regime-specific mean return

**switching_variance=True**

- allows each regime to have its own variance

**search_reps=10**

- tries multiple starting values / starting searches

# Cross-Check: Markov Switching Regression

0 - Bear

1 - Neutral

2 - Bull

```
==================================================================
                 coef    std err          z      P>|z|
------------------------------------------------------------------
const          -0.0175      0.012     -1.507      0.132
sigma2          0.0052      0.001      3.712      0.000
                            Regime 1 parameters
==================================================================
                 coef    std err          z      P>|z|
------------------------------------------------------------------
const           0.0001      0.001      0.094      0.925
sigma2          0.0007   5.33e-05     13.592      0.000
                            Regime 2 parameters
==================================================================
                 coef    std err          z      P>|z|
------------------------------------------------------------------
const           0.0032      0.000      7.106      0.000
sigma2          0.0002   1.18e-05     15.953      0.000
                         Regime transition parameters
==================================================================
Transition Prob  coef    std err          z      P>|z|
------------------------------------------------------------------
p[0->0]         0.8422        nan        nan        nan
p[1->0]         0.0099      0.006      1.576      0.115
p[2->0]       3.02e-06      0.002      0.001      0.999
p[0->1]         0.1578      0.094      1.672      0.094
p[1->1]         0.9602      0.013     73.212      0.000
p[2->1]         0.0196      0.008      2.496      0.013
------------------------------------------------------------------
```

p - Not Significant

p - Not Significant

p – Significant & Positive

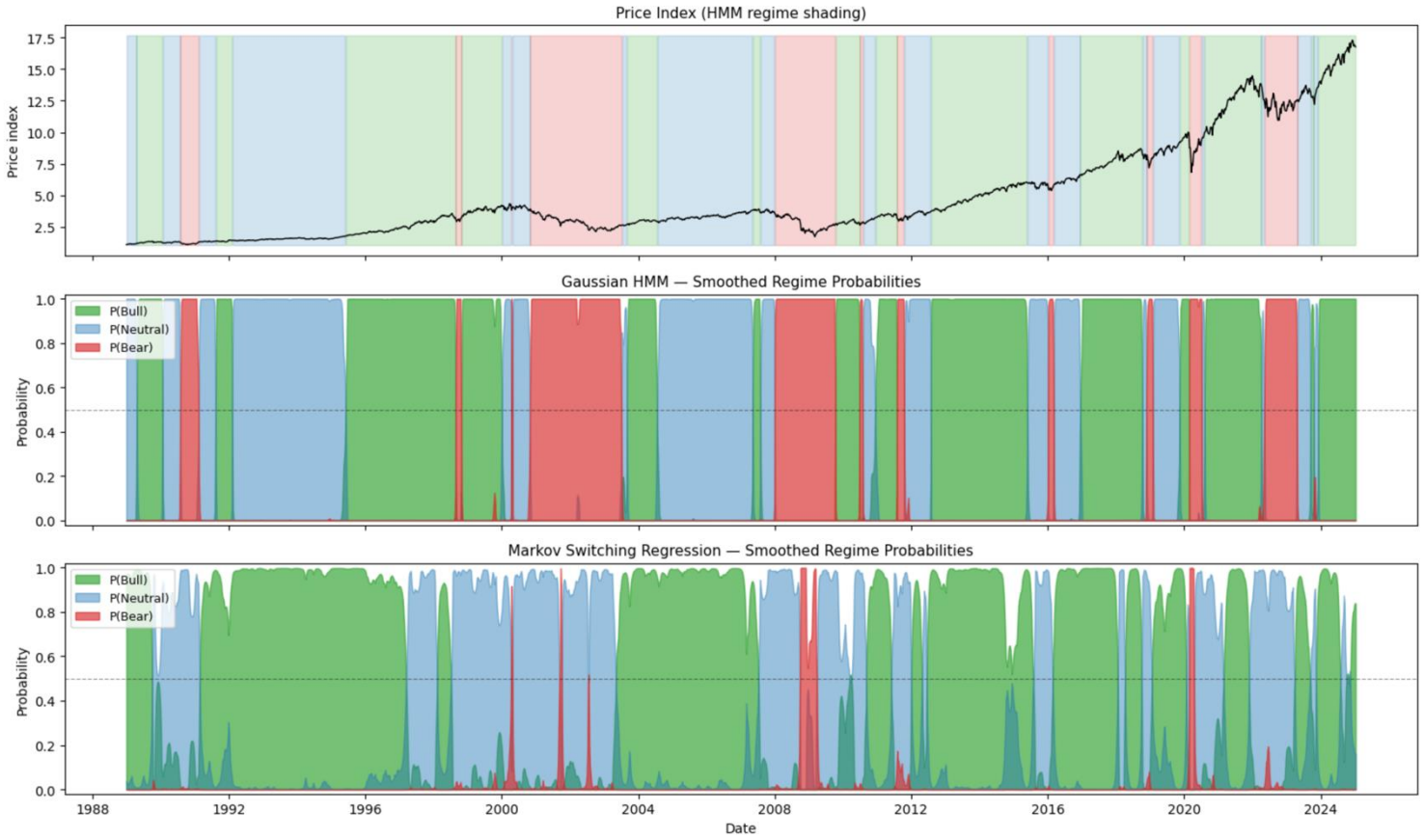
| Regime | Mean per week | Variance |
|---|---|---|
| Bear | -1.75% | 0.0052 ↑ |
| Neutral | 0. 01% | 0.0007 ≈ |
| Bull | 0.32% | 0.0002 ↓ |

| Regime | Persistence | Expected Duration | Approx. Duration |
|---|---|---|---|
| **Bear** | 0.8422 | 1/(1-0.8422) | **6.3 weeks** |
| **Neutral** | 0.9602 | 1/(1-0.9602) | **25.1 weeks** |
| **Bull** | 0.9804 | 1/(1-0.9804) | **51.0 weeks** |

| **Persistence** - probability that the process **remains in the same inferred regime from one period to the next** | | | |
|---|---|---|---|
| **Bear** | **0.8422** | 0.1578 | 0.0000 |
| **Neutral** | 0.0099 | **0.9602** | 0.0299 |
| **Bull** | 0.0000 | 0.0196 | **0.9804** |

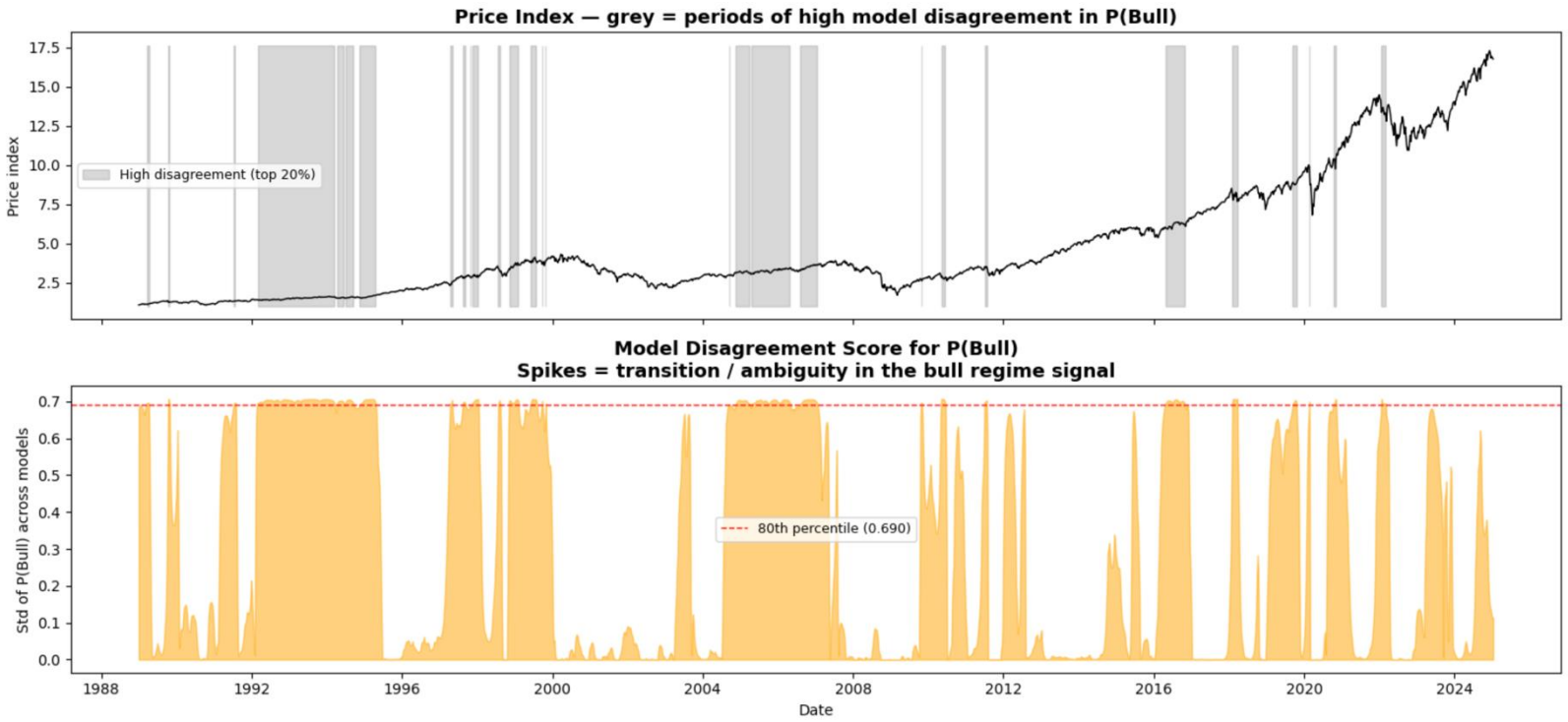
- MSR also finds a credible 3-state structure: **bear = negative and volatile**, **neutral = near zero**, **bull = positive and calm**, with persistent state dynamics

# HMM vs Markov Regression — Cross-Check & Consensus Forecast



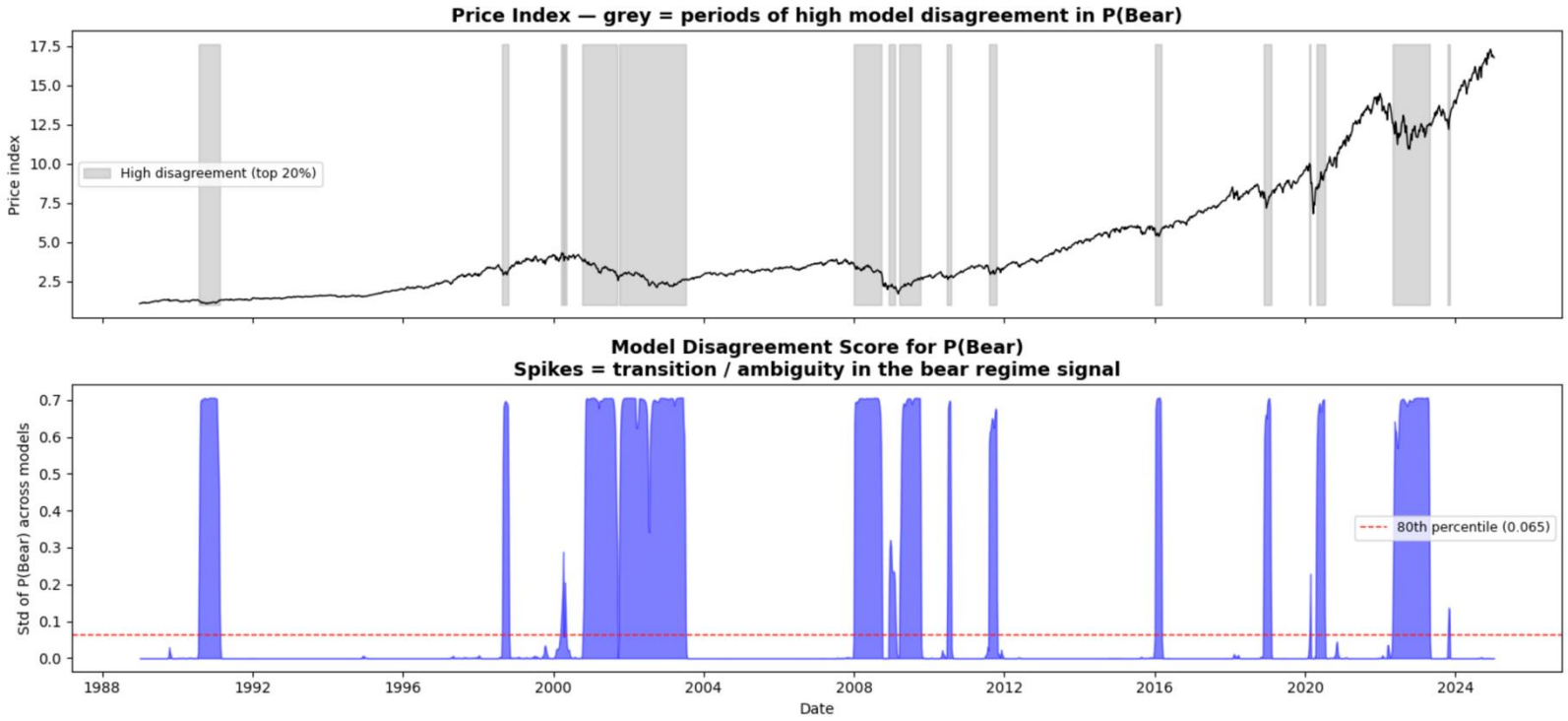

- Two different models, built differently and estimated differently, point to the same state pattern
- We can gain confidence that the regime structure is genuinely present in the data
- Disagreement is not necessarily a problem, it is often the most interesting part
- a) Market is near a **transition boundary,** b) one model reacts earlier than the other the signal is **ambiguous or mixed**
- Diverging models, might indicate market is between regimes, regime is weakly defined rather than clearly dominant

# HMM vs Markov Regression — Cross-Check & Consensus Forecast – p_Bull




- Grey bands mark dates when HMM–Markov disagreement in P(Bull) is higher than on about 80% of all dates in the sample.
- The red dashed line is the threshold for high disagreement: dates above it belong to the highest 20% of disagreement values in the sample

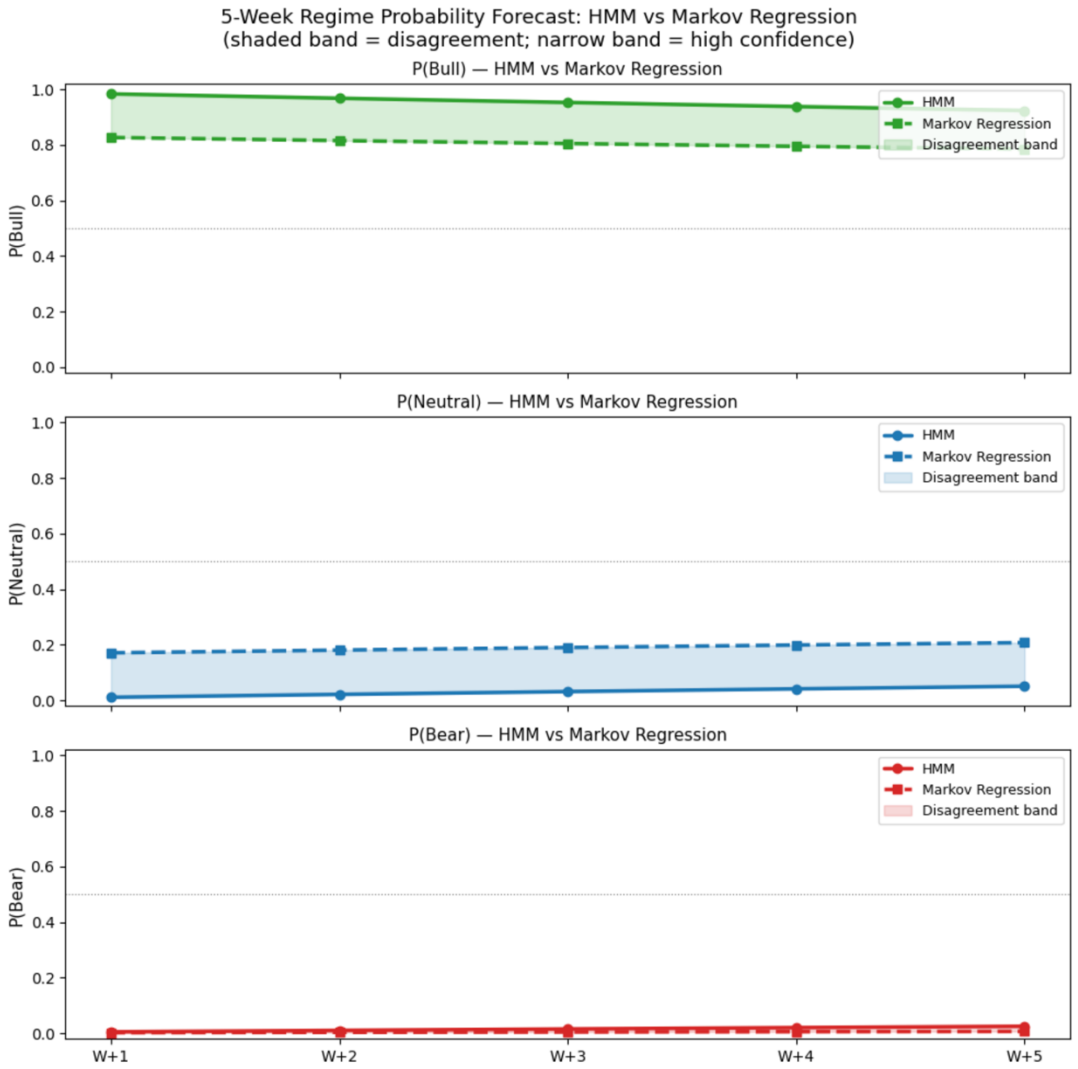
# HMM vs Markov Regression — Cross-Check & Consensus Forecast – p_Bear




- p_Bear does seem to show more alignment overall between the two models than p_Bull
- Relative to p_Bull, the p_Bear plot suggests tighter cross-model alignment for most of the sample, with disagreements concentrated mainly around regime-transition windows

# Combined 5-Week Forecast — HMM vs Markov Regression

**Markov Regression forecast is not forecasting returns but using  state-probability propagation** as the HMM, using the **Markov Switching Regression transition matrix** and the **last smoothed posterior state probabilities** from the regression model




**Forecast -** Conditional on the current estimated regime mix using

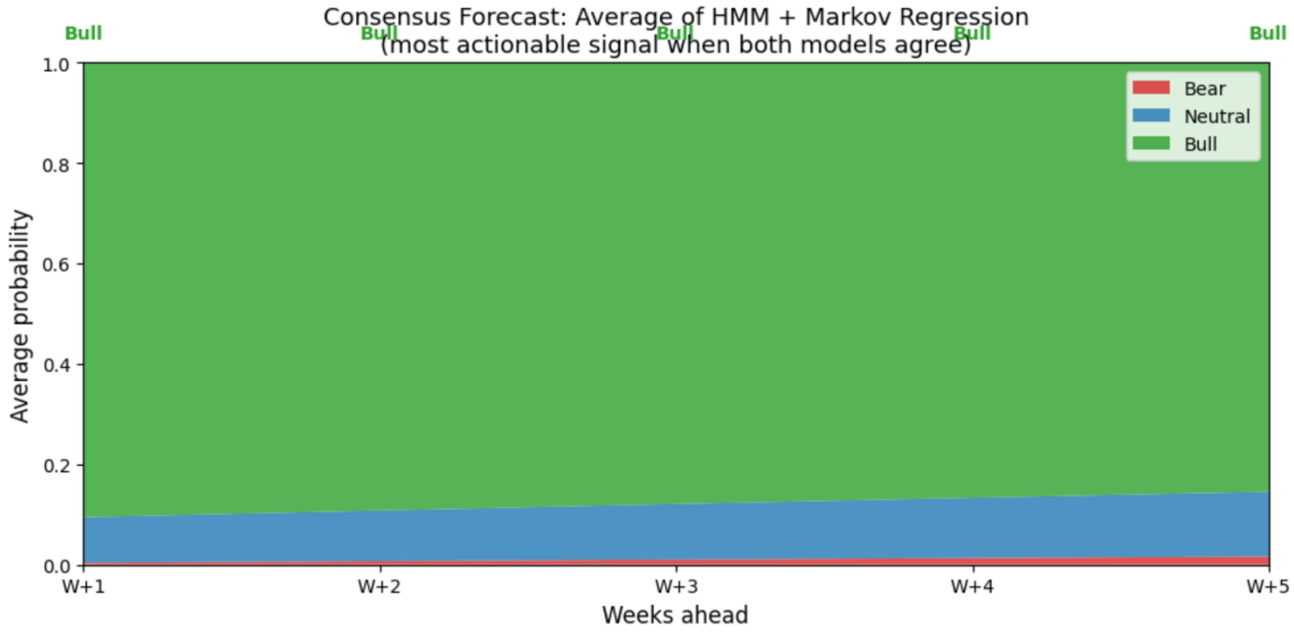
- Regime-specific means
- Regime-specific variances
- Markov switching dynamics
- Driven only by transition dynamics
- No new return observations entering after the last date

**Plot**

- p_Bull : Both models expect the bull regime to remain dominant, HMM expresses stronger conviction than the Markov Regression.
- p_Neutral : Markov Regression assigns more residual probability to a neutral state
- p_Bear : Both models strongly agree that a near-term bear regime is unlikely over the 5-week horizon

# Combined 5-Week Forecast — HMM vs Markov Regression - Consensus

- The **consensus** outlook remains **Bull-dominant over the next 5 weeks**, but with **gradually declining**
- Probability **shifts mainly from Bull toward Neutral**, while **Bear risk remains very low**




| Horizon | Dominant Regime | Bull % | Neutral % | Bear % |
|---|---|---|---|---|
| W+1 | Bull | 90.5% | 9.1% | 0.4% |
| W+2 | Bull | 89.2% | 10.1% | 0.7% |
| W+3 | Bull | 87.9% | 11.1% | 1.0% |
| W+4 | Bull | 86.7% | 12.0% | 1.3% |
| W+5 | Bull | 85.5% | 12.9% | 1.6% |

**Summary:**

- Regime-switching models can be used to turn raw market returns into an interpretable state framework for decision-making
- Rather than assuming one stable return-generating process, it treats markets as evolving through latent states

# Outcomes

- **Conceptual: markets are not stationary**
  - Students see why a single-regime model misleads in risk and return
- **Model validation: never trust a single model**
  - Cross-checking HMM with Markov Regression allows for triangulation
  - Agreement = confidence
  - Divergence = investigate further